

React.js

React is a **JavaScript library developed by Facebook** for building fast, interactive, and component-based user interfaces, especially for single-page web applications (SPAs).

Features of React

1. **Component-Based Architecture** – UI is divided into reusable components that manage their own structure and logic.
2. **Virtual DOM** – React updates only the changed parts of the UI using a fast in-memory representation of the DOM.
3. **Declarative Syntax** – You describe what the UI should look like, and React handles the updates automatically.
4. **JSX (JavaScript XML)** – JSX allows writing HTML-like code inside JavaScript for easier and cleaner component creation.
5. **One-Way Data Flow** – Data flows in a single direction (parent → child), making the application predictable.
6. **Reusable Components** – Components can be reused across the app, reducing duplicate code.
7. **Fast Performance** – Virtual DOM and efficient diffing algorithms make React UI updates extremely fast.
8. **Large Ecosystem** – Rich ecosystem with libraries for routing, state management, API handling, testing, etc.
9. **Strong Community Support** – Huge community, documentation, and third-party tools make development easier.
10. **Cross-Platform Development** – With React Native, the same approach can build mobile apps for Android/iOS.

How to Install React

Option 1: Install Using Create React App (CRA) – Beginner Friendly

```
npx create-react-app my-app  
cd my-app  
npm start
```

Creates a full project setup

Starts the development server on **http://localhost:3000**

Option 2: Install Using Vite (Fastest and Recommended)

1. Create project:

```
npm create vite@latest my-app --template react
```

2. Navigate & install dependencies:

```
cd my-app  
npm install
```

3. Start the dev server:

```
npm run dev
```

React Project Folder Structure (Standard)

Below is the **default folder structure** when you create a React project:

```
my-app/  
├── node_modules/      → Contains all installed packages  
├── public/            → Static files (index.html, images, icons)  
│   └── index.html     → Main HTML file where React injects UI  
├── src/               → Main folder where your code lives  
│   ├── assets/        → Images, icons, static files  
│   ├── components/    → Reusable UI components (Header, Navbar, Card)  
│   ├── pages/         → Page-level components (Home, About, Login)  
│   ├── App.js         → Root component of the project  
│   ├── index.js       → ReactDOM renders App component here  
│   └── styles/        → CSS files (optional custom folder)  
├── package.json       → Project information + dependencies  
├── package-lock.json  → Exact version lock for dependencies  
├── .gitignore         → Files to ignore in Git  
└── README.md         → Project documentation
```

Simple Explanation

- **src/** → You write your React code here
- **public/** → React UI loads inside **index.html**
- **App.js** → Main component
- **index.js** → Starts the app
- **components/** → Reusable UI parts

What is JSX?

JSX (JavaScript XML) is a **syntax extension for JavaScript** that allows you to write HTML-like code **inside JavaScript**.

It makes React UI code simpler, cleaner, and easier to understand.

JSX Example:

```
function Welcome() {  
  return <h1>Hello React!</h1>;  
}
```

Here,

`<h1>Hello React!</h1>` looks like HTML but **it's not real HTML** — it's **JSX**.

How JSX Works Internally

Even though JSX looks like HTML, **React converts JSX into JavaScript** using Babel.

JSX is converted like this:

JSX

```
<h1>Hello</h1>
```

Converted JavaScript

```
React.createElement("h1", null, "Hello");
```

So JSX is simply a **developer-friendly way to create React elements**.

Why JSX is Used?

1. Makes UI code readable
2. Mix HTML layout with JavaScript logic
3. Easier to debug
4. React recommends it for cleaner component structure

JSX Rules

Return only one parent element

```
return (  
  <div>  
    <h1>Hi</h1>  
    <p>Welcome</p>  
  </div>  
);
```

Use camelCase attributes

- class → className
- onclick → onClick

JavaScript inside JSX must be in { }

```
<h1>{name}</h1>
```

JSX must be closed properly

```
<input />  
<img />
```

JS vs JSX in React

Point	JavaScript (JS)	JSX (JavaScript XML)
Definition	Standard programming language used for logic and DOM operations	Syntax extension that allows writing HTML-like code in JavaScript
DOM Handling	Manual DOM manipulation	React updates DOM automatically
Syntax Style	Pure JavaScript syntax	HTML-like syntax integrated inside JS
Readability	Less readable for UI creation	Very readable, looks like HTML
UI Creation	Uses <code>React.createElement()</code>	Directly writes UI with <code><div></code> , <code><h1></code> , etc.
Mixing Logic & UI	Harder; must concatenate or build manually	Easy; use <code>{ }</code> to embed JS inside UI
Learning Curve	Simple to learn	Slight learning curve due to JSX rules
Browser Support	Runs directly in browsers	Needs Babel to transform into JS
Attributes	Uses JS attribute names	Uses React-specific names (e.g., <code>className</code> , <code>htmlFor</code>)
Closing Tags	No need for closing elements	All tags must be properly closed
Usage in React	Mostly for logic and backend code	Recommended for UI creation in components

Components in React

A **Component** in React is a reusable, independent piece of UI that renders part of the webpage. Components allow you to **break the UI into small, reusable pieces** and manage each one separately.

Think of components like **LEGO blocks**—each block is small, but together they build a full UI.

Why Components Are Used?

- To reuse UI parts
- To make code clean and modular
- To make UI easier to maintain
- To separate logic and UI into pieces
- To improve readability and scalability

Types of Components in React

React mainly has **two types** of components:

- **Functional Components**
- **Class Components**

Functional Components (Modern & Recommended)

A **Functional Component** is a simple JavaScript function that returns JSX.

Features

- Simpler and easy to write
- Fast and lightweight
- Use **Hooks** (like `useState`, `useEffect`)
- Most commonly used in modern React

Example

```
function Hello() {  
  return <h1>Hello from Functional Component</h1>;  
}  
  
export default Hello;
```

Class Components (Older, Not Common Now)

A **Class Component** is a JavaScript class that extends **React.Component** and uses a **render()** method.

Features

- More complex
- Used before hooks were introduced
- Supports lifecycle methods (**componentDidMount**, etc.)
- Still works but not recommended for new apps

Example

```
import React, { Component } from "react";  
  
class Hello extends Component {  
  render() {  
    return <h1>Hello from Class Component</h1>;  
  }  
}  
  
export default Hello;
```

Functional vs Class Components

Feature	Functional Component	Class Component
Syntax	Function	Class
Hooks Support	Yes	No (only lifecycle methods)
Readability	Easy	Complex
Performance	Fast	Slightly slower
Lifecycle Handling	via Hooks	via methods
Common Usage	Very high	Rare now
Code Length	Short	Long

Data Passing in Components (Props)

In React, **data is passed from a parent component to a child component using Props.**

Props = Properties (Read-Only Data)

- Props allow components to communicate.
- Props make components dynamic.
- Props are **immutable** (cannot be changed by the child).

Parent → Child Data Passing (Most Common)

Parent Component

```
function App() {  
  return <Student name="Shubham" age={22} />;  
}
```

Child Component

```
function Student(props) {  
  return (  
    <h2>Name: {props.name}, Age: {props.age}</h2>  
  );  
}
```

Parent sends → Child receives via **props**.

Child → Parent Data Passing (Using Callback Function)

Parent

```
function App() {  
  const getData = (value) => {  
    console.log("Child sent:", value);  
  };  
  
  return <Child sendData={getData} />;  
}
```

Child

```
function Child(props) {  
  return (  
    <button onClick={() => props.sendData("Hello Parent")}>  
      Send to Parent  
    </button>  
  );  
}
```

Child calls a **function passed as prop** to send data upward.

Sibling → Sibling Data Passing

- Done through a **common parent**
- Parent shares data to both children
- One child can send data via callback

Properties of Data Passing (Properties of Props)

Here are the **key properties** (each in simple one-line form):

1. **Props are read-only** – Child cannot modify them.
2. **Props enable component communication** – Mainly parent → child.
3. **Props make components dynamic** – Same component works with different data.

4. **Props can pass any type of data** – Strings, numbers, arrays, objects, functions.
5. **Props ensure one-way data flow** – Data always moves downward (top to bottom).
6. **Props help create reusable UI** – Components can work with variable inputs.
7. **Props are passed like function arguments** – Simple and clean syntax.
8. **Props trigger re-render when changed** – If parent updates data, UI updates.

Reusable Components in React

A **Reusable Component** is a component that can be used multiple times with different data.

Why Reusable Components?

- Reduce duplicate code
- Improve maintainability
- Improve consistency of design
- Make apps scalable

Example: Reusable Button Component

Button Component (Reusable)

```
function Button({ label, color }) {  
  return (  
    <button style={{ backgroundColor: color }}>  
      {label}  
    </button>  
  );  
}  
  
export default Button;
```

Usage

```
<Button label="Save" color="green" />  
<Button label="Delete" color="red" />  
<Button label="Update" color="blue" />
```

→ Same UI, different labels and styles.

Example: Reusable Card Component

Card Component

```
function Card({ title, description }) {  
  return (  
    <div className="card">  
      <h2>{title}</h2>  
      <p>{description}</p>  
    </div>  
  );  
}
```

Usage

```
<Card title="React" description="A JavaScript library for UI" />  
<Card title="Node.js" description="JavaScript runtime environment" />  
<Card title="MongoDB" description="NoSQL database" />
```

→ One component → multiple uses

State in React

State is a built-in React object that stores **dynamic data** of a component and allows the UI to **change** when the data changes.

In simple words:

- State is a variable that React watches.
- When the state changes → the UI automatically updates.

Key Points About State

- State belongs **only to the component** where it is created.
- State is **mutable** — it can be changed using **setState** (class components) or **setState function** (**useState** in functional components).
- State updates are **asynchronous**.
- When state changes → **component re-renders**.

useState Hook

```
import React, { useState } from "react";
function App() {
  const [name, setName] = useState("Shubham");
  return (
    <div>
      <h2>My Name: {name}</h2>
      <button onClick={() => setName("Rahul")}>
        Change Name
      </button>
    </div>
  );
}
export default App;
```

Explanation:

- name → current state value
- setName → function to update state
- UI updates when setName("Rahul") is called

Why Do We Use State?

- To store **changing data**
- To handle **user inputs**
- To update **UI on events** (click, typing, submit)
- To maintain **component behavior** (toggle, open/close, counter)

Common Examples of State

Example	Stored as State
Counter	number
Form input	string
Toggle show/hide	boolean
API response data	object/array

Building Forms in React.js

In React, forms are built using **controlled components**, meaning form inputs are connected to **state**.

The form input value is stored in **state**, and any change in input updates that state.

Steps to Build Forms Using State

1. **Create state** for each form field

```
const [name, setName] = useState("");
```

2. **Bind the input value to state**

```
<input value={name} />
```

3. **Listen to input change**

```
<input onChange={(e) => setName(e.target.value)} />
```

4. **Handle form submit**

Use **onSubmit** event to process form data.

Why Controlled Forms?

- React controls the data, not the DOM
- Easy validation
- Easy to reset fields
- Easy to handle multiple fields

```
import React, { useState } from "react";

function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
  }
}
```

```

    alert(`Name: ${name} | Email: ${email}`);
    setName("");
    setEmail("");
  };

  return (
    <div style={{ padding: "20px" }}>
      <h2>React Form with State</h2>

      <form onSubmit={handleSubmit}>
        <div>
          <label>Name: </label>
          <input
            type="text"
            value={name}
            onChange={(e) => setName(e.target.value)}
          />
        </div>

        <br />

        <div>
          <label>Email: </label>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
          />
        </div>

        <br />

        <button type="submit">Submit</button>
      </form>
    </div>
  );
}

export default App;

```

Routing in React

Routing in React is handled by a library called **React Router**.

It allows you to create **multiple pages** inside a single-page application (SPA) without reloading the browser.

React Router helps you move between pages (Home, About, Contact) without refreshing the page.

Features of React Router

- Enables navigation between pages
- Uses URLs to change views
- No full page reloads (SPA behavior)
- Supports nested routes
- Supports dynamic routes (ex: `/user/10`)
- Provides navigation components (Link, NavLink)
- Allows programmatic navigation (useNavigate)

Installation

Run this inside your React project:

```
npm install react-router-dom
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { BrowserRouter } from "react-router-dom";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

App.js

```
import React from "react";
import { Routes, Route, Link } from "react-router-dom";
function Home() {
  return <h2>Home Page</h2>;
}
function About() {
  return <h2>About Page</h2>;
}
function App() {
  return (
    <div>
      <h1>React Router Example</h1>
      { /* Navigation */ }
      <nav>
        <Link to="/">Home</Link> |
        <Link to="/about">About</Link>
      </nav>
      { /* Routing */ }
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </div>
  );
}
export default App;
```

How Routing Works

Concept	Meaning
<BrowserRouter>	Enables routing in the app
<Routes>	Collection of all routes
<Route>	Single route definition
path="/"	URL of the route
element={<Home />}	Component to show
<Link>	Navigation without refresh

React Flux

Flux is **an architecture pattern** used in React applications to manage **unidirectional data flow**.

Flux helps manage **complex state** by ensuring data always flows in **one direction** → making apps more predictable.

Why Flux?

- Keeps data flow **one-way**, so debugging is easy
- Avoids messy two-way binding
- Good for **large applications with many components**
- Ensures state is updated through a **central store**

Flux Architecture

Flux has **4 main parts**:

1. **Actions**
 - Events triggered by user or system
 - Example: *Add a product, update profile, delete item*
2. **Dispatcher**
 - The central hub that **dispatches actions** to stores
3. **Stores**
 - Holds the **application state**
 - Updates state when dispatcher sends an action
4. **View (React Components)**
 - Re-render based on updated store data

Flow Diagram

```
User Action → Action → Dispatcher → Store → View (React)
```

Flux is not commonly used now—**Redux replaces Flux** with a cleaner pattern.

Flux Installation

Run this inside your React project:

```
npm install flux
```

After installation, you can import Flux modules like:

```
import { Dispatcher } from "flux";
```

EXAMPLE :

```
import React, { useState, useEffect } from "react";
import { Dispatcher } from "flux";
import { EventEmitter } from "events";

// ----- Dispatcher -----
const AppDispatcher = new Dispatcher();

// ----- Actions -----
const TodoActions = {
  addTodo(text) {
    AppDispatcher.dispatch({
      type: "ADD_TODO",
      text,
    });
  },
};

// ----- Store -----
const TodoStore = new EventEmitter();
let todos = [];

AppDispatcher.register((action) => {
  if (action.type === "ADD_TODO") {
    todos.push(action.text);
    TodoStore.emit("change");
  }
});
```

```

TodoStore.getTodos = () => todos;

// ----- View Component -----
function App() {
  const [list, setList] = useState(TodoStore.getTodos());
  const [input, setInput] = useState("");

  useEffect(() => {
    TodoStore.on("change", () => {
      setList([...TodoStore.getTodos()]);
    });
  }, []);

  const addItem = () => {
    if (input.trim() === "") return;
    TodoActions.addTodo(input);
    setInput("");
  };

  return (
    <div style={{ padding: "20px" }}>
      <h2>Flux Short Example</h2>

      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        placeholder="Add item"
      />
      <button onClick={addItem}>Add</button>

      <ul>
        {list.map((t, i) => (
          <li key={i}>{t}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;

```

Babel

Babel is a **JavaScript compiler** that converts **modern JavaScript + JSX** into **older JavaScript** that all browsers understand.

Babel allows you to write **JSX** and **ES6+ features**, and converts them into compatible JavaScript.

Why Do We Need Babel?

- Browsers **do not understand JSX**
- Older browsers **do not support ES6+** (let, const, classes, arrow functions)
- Babel makes your code **safe + compatible**
- Used internally by React frameworks like:
 - Create React App
 - Vite
 - Next.js

JSX Example

```
const element = <h1>Hello React!</h1>;
```

Browsers DO NOT understand this.

What Babel Converts It To (Browser Understands)

```
const element = React.createElement("h1", null, "Hello React!");
```

- No JSX
- Pure JavaScript
- Every browser supports it

What Babel Can Compile

Modern Feature	Example
JSX	<code><div>Hello</div></code>
Arrow Functions	<code>() => {}</code>
Classes	<code>class App {}</code>
Spread operator	<code>{ ...data }</code>
Optional chaining	<code>user?.name</code>
Template strings	<code>Hello \${name}</code>

How Babel Works in React Projects

In **create-react-app**, Babel is already installed:

```
npx create-react-app myapp
```

This setup includes:

- Babel
- Webpack
- React Presets
- JSX support

No manual configuration needed.

If You Want to Install Babel Manually

(For custom React setups)

Install core packages:

```
npm install @babel/core @babel/cli @babel/preset-env @babel/preset-react
```

Create babel.config.json:

```
{
  "presets": [ "@babel/preset-env", "@babel/preset-react" ]
}
```

This tells Babel to compile:

- ES6+ features (**preset-env**)
- JSX for React (**preset-react**)

Simple Example Using Babel CDN

Create an HTML file:

```
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

<div id="root"></div>

<script type="text/babel">
  function App() {
    return <h1>Hello from JSX!</h1>;
  }

  ReactDOM.render(<App />, document.getElementById("root"));
</script>
```

- Babel compiles JSX
- Works in browser directly